

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T08:53:33.5380527Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260924-085333540.prof
Profil mémoire	memory-20260924-085333540.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	4.59 ms	4594383.6	40187.2	17

Détail hyperfine

Métrique	Valeur
Moyenne	4.3343 s
Écart-type	0.0793 s
Médiane	4.3422 s
Temps utilisateur	19.6931 s
Temps système	1.0800 s
Min	4.2464 s
Max	4.4163 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	20.41s
Mémoire allouée (pprof)	85.24MB
Mémoire par opération	40187.2 B/op

Garbage collector Go

Cycles GC	65
Temps GC cumulé	57.310 ms

Plus longue pause	4.000 ms
Temps CPU GC cumulé	313.570 ms
Objectif mémoire maximal	95.00 MB

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **20.41s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).infectionCandidate	16.72s	81.92%	18.83s	92.26%
jeu-de-la-vie/game.(*Board).index (inline)	1.69s	8.28%	1.70s	8.33%
jeu-de-la-vie/game.(*Board).Step	0.83s	4.07%	1.28s	6.27%
jeu-de-la-vie/game.squaredDistance (inline)	0.25s	1.22%	0.25s	1.22%
jeu-de-la-vie/game.(*Board).countInfectionCand	0.24s	1.18%	19.07s	93.43%
math/rand.(*Rand).Float64 (inline)	0.24s	1.18%	0.44s	2.16%
runtime.asyncPreempt	0.17s	0.83%	0.17s	0.83%
math/rand.(*rngSource).Uint64 (inline)	0.12s	0.59%	0.12s	0.59%

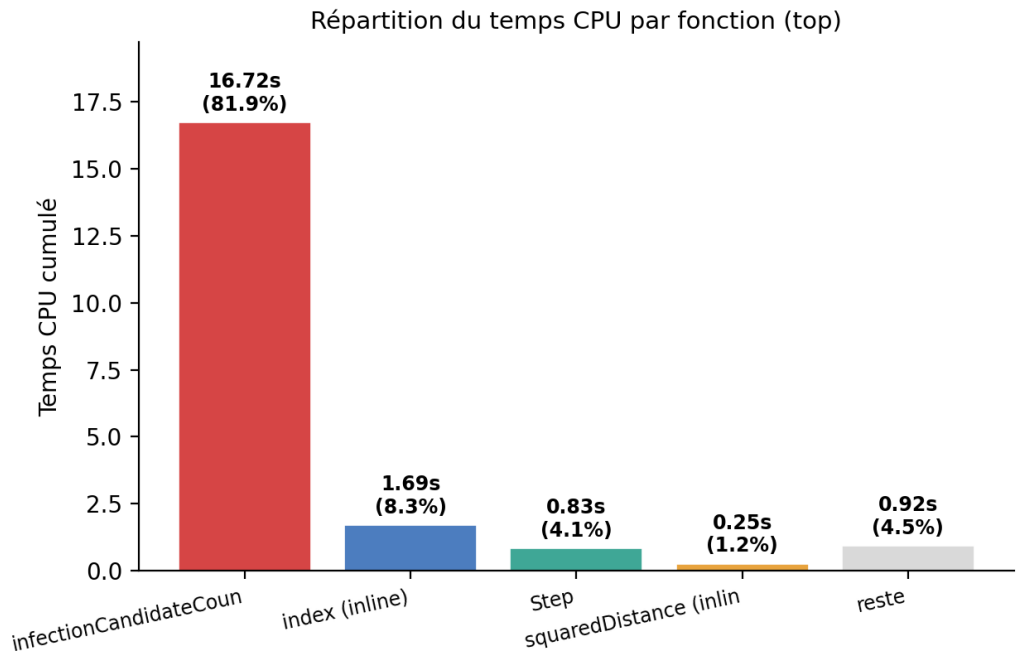


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

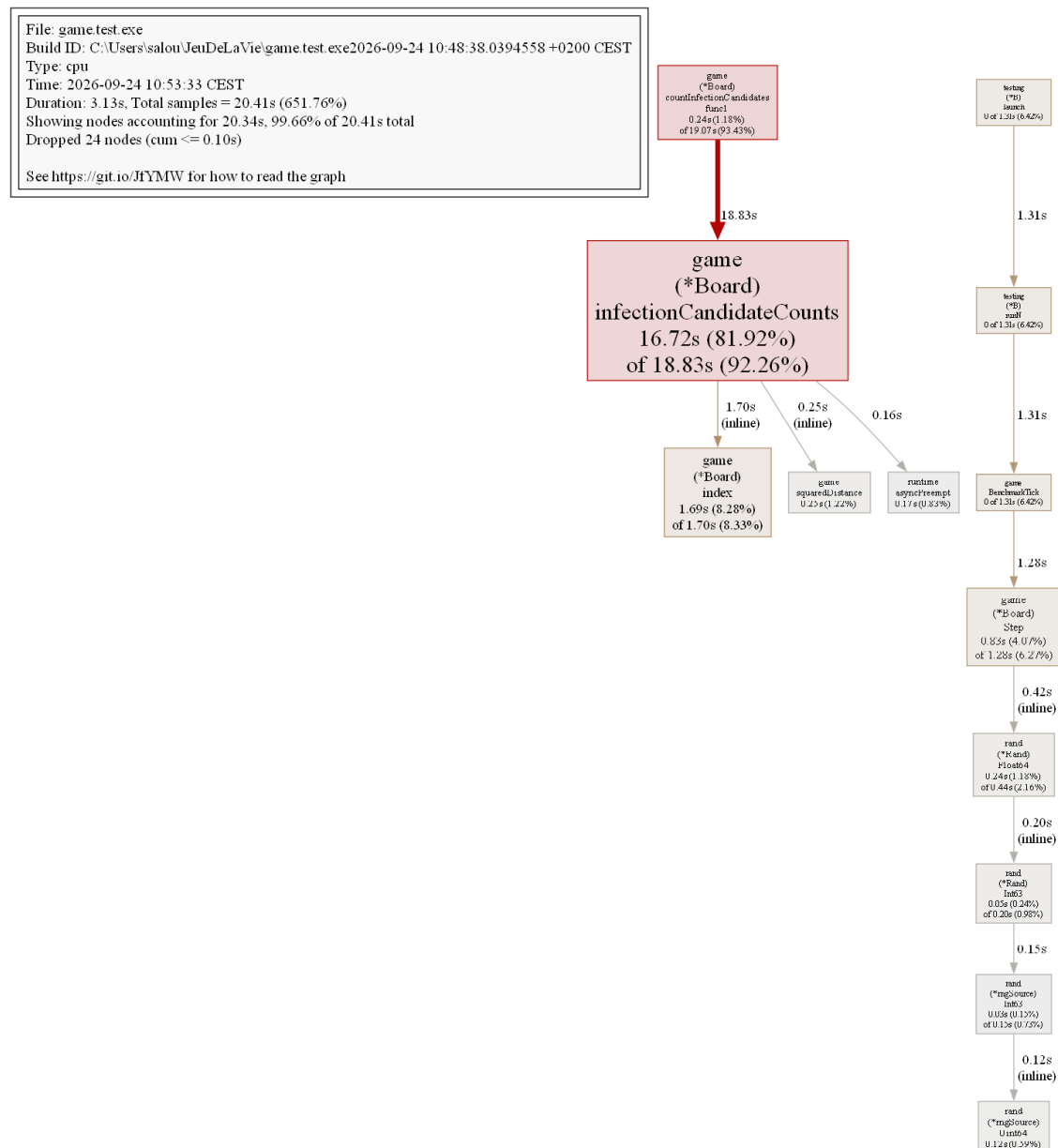


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).infectionCandidateCounts**, responsable de **81.9%** du temps CPU total (flat), pour **92.3%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

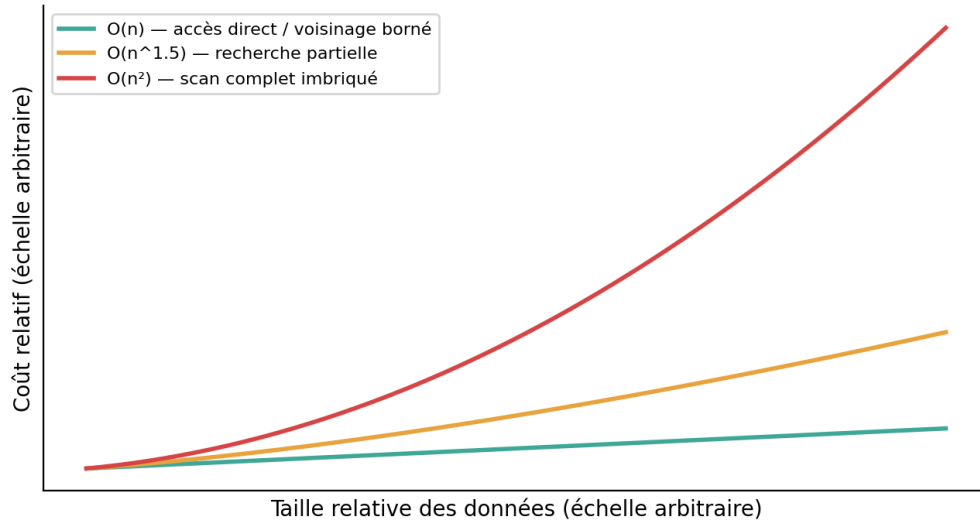


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **85.24MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).ensureScratch (inl	42.11MB	49.40%	42.11MB	49.40%
jeu-de-la-vie/game.NewBoard (inline)	17.95MB	21.06%	17.95MB	21.06%
jeu-de-la-vie/game.RandomBoard	13.81MB	16.20%	31.76MB	37.26%
runtime.mallocgc	3.51MB	4.11%	3.51MB	4.11%
jeu-de-la-vie/game.(*Board).countInfectionCand	2MB	2.35%	2MB	2.35%
runtime/pprof.StartCPUProfile	1.72MB	2.02%	1.72MB	2.02%
compress/flate.(*compressor).init	0.64MB	0.75%	0.64MB	0.75%
math/rand.newSource (inline)	0.50MB	0.59%	0.50MB	0.59%

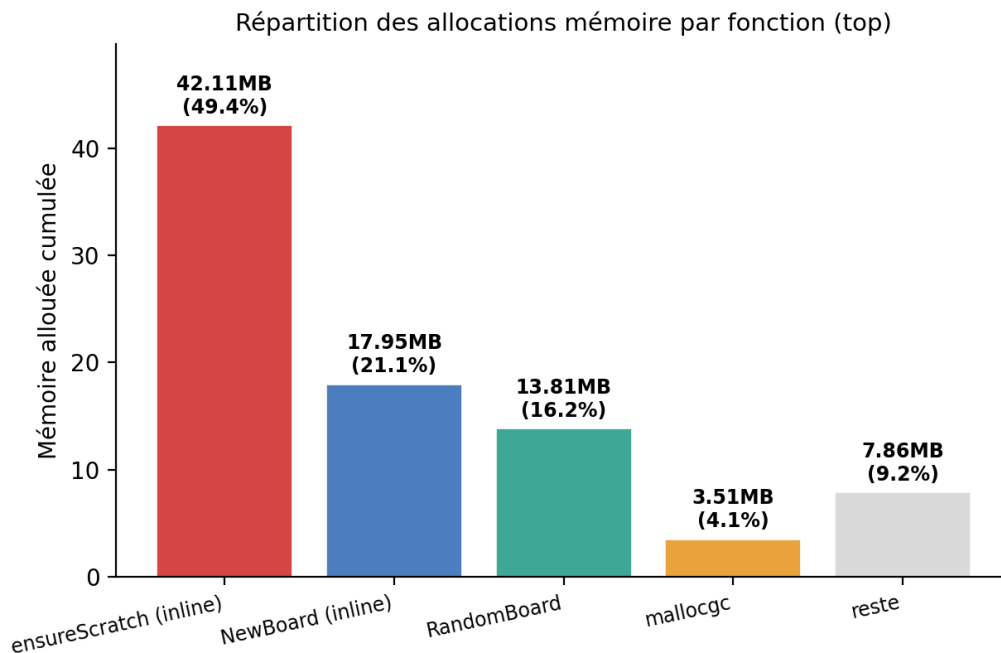


Figure — Répartition des allocations mémoire par fonction.

File: game.test.exe
 Build ID: C:\Users\salou\JeuDeLaVie\game.test.exe2026-09-24 10:48:38.0394558 +0200 CEST
 Type: alloc_space
 Time: 2026-09-24 10:53:36 CEST
 Showing nodes accounting for 77.38MB, 90.78% of 85.24MB total
 Showing top 9 nodes out of 79
 See <https://git.io/JfYMW> for how to read the graph

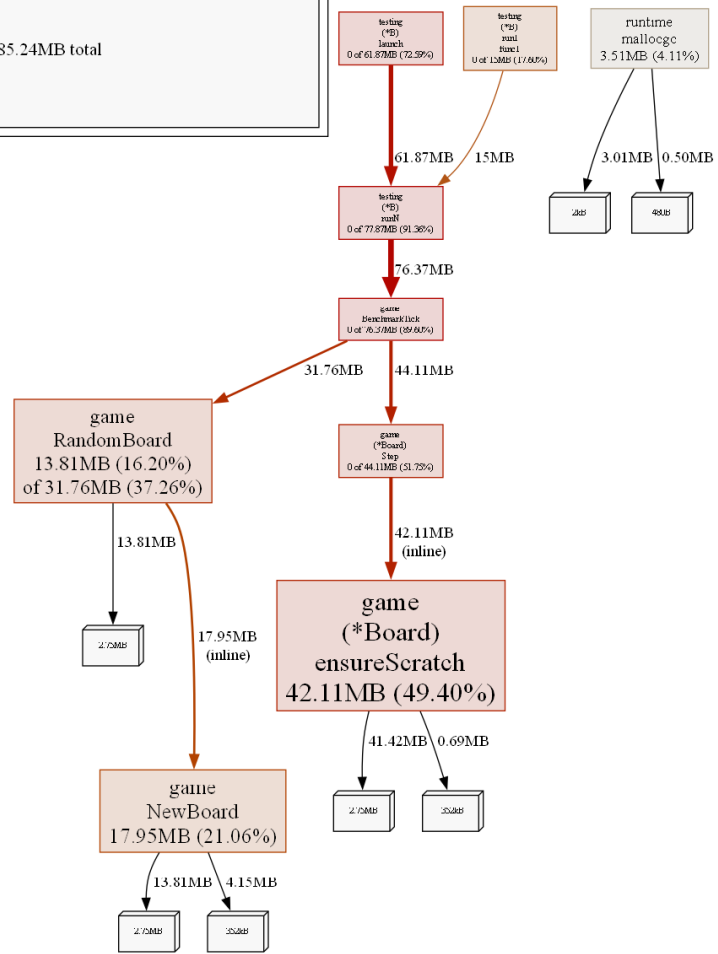


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 10:54:12 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.